

GENTLENESS

March 13, 2026

Designing a Dynamic Semantic Cache for LLM-Based Access Control

Leveraging Large Language Models for Context-Aware Authorization Decisions

1 Introduction

It is widely recognized that some of the data collected, processed and stored in current information systems is of a personal nature, making it sensitive and requiring appropriate protection. Organizations, whether public or private, e.g., in domains like healthcare, finance, logistics that handle such data are consequently subject to increasingly stringent regulations. It is then crucial to equip information systems with mechanisms that control and restrict access to sensitive data to authorized users only. While experts in cybersecurity are involved in the design of advanced mechanisms to ensure data protection, an increasing number of individuals and organizations with limited technical expertise are tasked with managing large volumes of sensitive data. In most cases, these non-expert users are actually more aware of the operational realities of their domain and often have a more accurate and detailed vision of the field. However, the design and implementation of security policies in the actual information systems remain complex tasks that typically require significant expertise in formal security models. This complexity creates a gap between non-expert stakeholders, who are able to express their security requirements in natural language or through informal specifications, and the technical mechanisms needed to enforce these requirements effectively. As a result, there is a risk that the intended security objectives are either misinterpreted or only partially implemented, leading to vulnerabilities or inconsistencies.

Traditional Access Control systems (RBAC, ABAC) rely on **static rules** and struggle with:

- Ambiguous requests (e.g., “Grant access if the requester is a ‘close relative’ of the patient”)
- Dynamic contexts (e.g., emergency scenarios)
- Scalability (recomputing complex rules for repeated requests)

2 Motivation

LLMs can address these gaps by:

- Interpreting natural language requests and generating semantic embeddings
- Comparing new requests against a semantic cache of past decisions
- Adapting to fuzzy contexts (e.g., “urgent”, “trusted colleague”)

3 Objectives

- Design a semantic cache that stores embeddings of access requests/contexts
- Reduce decision latency for repeated requests
- Improve accuracy for ambiguous requests
- Detect anomalies (e.g., cache poisoning attacks)

4 System Design

4.1 Inputs

Table 1: System Inputs

Type	Example	Format
Access Request	"I'm the on-call nurse; I need patient X's medical record."	Natural language text
Context	Time: 3:00 AM, Location: ER, Role: Nurse, History: 5 prior approved accesses	JSON/Structured
Access Rules	"Grant if (role = health-care_professional AND time > 10PM AND time < 6AM)"	Formal logic (XACML)
Feedback	Audit logs: "Access denied at 3:05 AM for record Y (reason: role mismatch)"	Structured (CSV/JSON)

4.2 Outputs

Table 2: System Outputs

Type	Example	Format
Decision	{"access": "granted", "confidence": 0.92, "reason": "emergency context validated"}	JSON
Cache Update	Store request embedding + decision + context	Vector DB (FAISS)
Alert	"Anomaly detected: High similarity to known privilege escalation attack (score: 0.87)"	Notification

4.3 Process Flow

1. Request Preprocessing

- Tokenize natural language request
- Enrich with structured context metadata
- Generate embedding using LLM (e.g., Mistral-7B)

2. Cache Lookup

- Compute cosine similarity with cached embeddings
- Apply confidence thresholds:
 - >0.9: Retrieve cached decision
 - 0.7-0.9: Submit to validation module
 - <0.7: Process as new request

3. Access Decision

- Check static rules (RBAC/XACML)
- Perform semantic analysis with LLM
- Combine scores (e.g., 70% static + 30% semantic)

4. Cache Update

- Store new embeddings with TTL
- Implement eviction policies (LRU/criticality-based)
- Update based on audit feedback

5. Anomaly Detection

- Compare against attack pattern cache
- Trigger alerts for high similarity scores

5 Technical Implementation

5.1 Technology Stack

Component	Tools/Technologies
LLM	Mistral-7B, Hugging Face Transformers, OpenAI API
Vector Database	FAISS, Weaviate, Pinecone
Access Control	Keycloak, OpenPolicyAgent
Backend	Python (FastAPI), Go
Evaluation	Latency/accuracy metrics, security testing

5.2 Deliverables

- Functional prototype with REST API
- Semantic cache implementation
- Monitoring dashboard
- Synthetic dataset with test scenarios
- Benchmarking results vs. traditional RBAC
- Final report with literature review

6 Extensions and Future Work

- Federated caching across organizations
- Explainable AI for decision justification
- Continuous learning from audit logs
- Integration with zero-trust architectures

7 Challenges and Mitigations

Risk	Description	Mitigation
Privacy Leaks	Embeddings may reveal sensitive information	Differential privacy, anonymization
Computational Cost	High resource usage for embedding generation	Hierarchical caching, batch processing
Adversarial Attacks	Crafted requests to exploit LLM vulnerabilities	Input sanitization, robust preprocessing
Cache Poisoning	Malicious entries corrupting the cache	Cryptographic verification, TTL policies